



100

Accessibility

These checks highlight opportunities to [improve the accessibility of your web app](#). Automatic detection can only detect a subset of issues and does not guarantee the accessibility of your web app, so [manual testing](#) is also encouraged.

ADDITIONAL ITEMS TO MANUALLY CHECK (10)

Hide

☐ Interactive controls are keyboard focusable



Custom interactive controls are keyboard focusable and display a focus indicator. [Learn how to make custom controls focusable](#).

☐ Interactive elements indicate their purpose and state



Interactive elements, such as links and buttons, should indicate their state and be distinguishable from non-interactive elements. [Learn how to decorate interactive elements with affordance hints](#).

☐ The page has a logical tab order



Tabbing through the page follows the visual layout. Users cannot focus elements that are offscreen. [Learn more about logical tab ordering](#).

☐ Visual order on the page follows DOM order



DOM order matches the visual order, improving navigation for assistive technology. [Learn more about DOM and visual ordering](#).

☐ User focus is not accidentally trapped in a region



A user can tab into and out of any control or region without accidentally trapping their focus. [Learn how to avoid focus traps](#).

☐ The user's focus is directed to new content added to the page



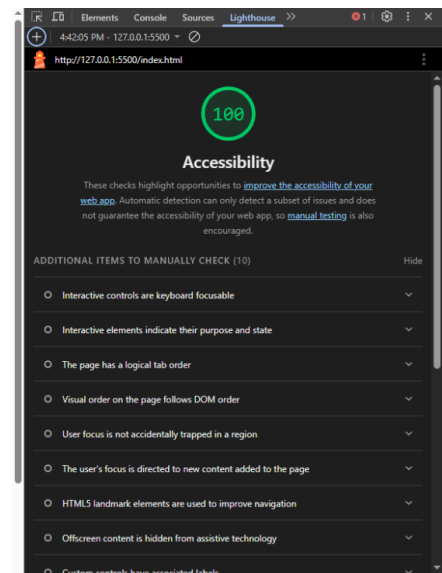
If new content, such as a dialog, is added to the page, the user's focus is directed to it. [Learn how to direct focus to new content](#).



QB PASSING ACADEMY

Reserve your spot now and save 50% when you register this week.

Sharpen your mechanics, footwork, and timing with position-specific drills led by experienced QB coaches. Each session focuses on reads, pocket movement, and



PASSED AUDITS (16)

Hide

`[aria-hidden="true"]` is not present on the document `<body>`



Assistive technologies, like screen readers, work inconsistently when `aria-hidden="true"` is set on the document `<body>`.
[Learn how aria-hidden affects the document body.](#)

`[role]`s have all required `[aria-*)` attributes



Some ARIA roles have required attributes that describe the state of the element to screen readers. [Learn more about roles and required attributes.](#)

Image elements have `[alt]` attributes



Informative elements should aim for short, descriptive alternate text. Decorative elements can be ignored with an empty alt attribute. [Learn more about the alt attribute.](#)

`[user-scalable="no"]` is not used in the `<meta name="viewport">` element and the `[maximum-scale]` attribute is not less than 5.



Pros

- Maps each issue to a specific **WCAG** rule with **severity/impact** labels.
- Gives “**Why it matters**” and “**How to fix**” guidance right in DevTools.
- Highlights the **exact DOM node**; you can jump to it in Elements to fix fast.
- **Fast scans**; filters by impact/type so you can triage quickly.
- Great for **regression checks** as you edit (rerun in seconds).
- Free version is enough for most class projects.

Cons

- **Email quirks**: may flag table-based patterns that are normal in HTML email (e.g., presentational tables).
- Won't catch **manual UX issues** (confusing alt text, poor focus order) unless you test them yourself.